Spring Expression: case study details

Arup Halder | Repository supplement | Published architecture version 1

A controlled dependency change passes functional tests but fails the architecture gate.

The study covers **spring-expression/src/main/java**, one module of Spring Framework. The same three-component architecture and one denied dependency rule were used for baseline, controlled violation, and repair.

Stage	Fork commit	Tests passed	Skipped	Edges	DCode gate	Violations
Baseline	30d8786e	1,074	11	516	Passed	0
Controlled violation	b9cf2ac1	1,077	11	517	Failed	1
Repair	3bce90df	1,074	11	516	Passed	0

All three stages had **zero functional test failures and errors** and the same **144 type/nested-type graph nodes**. Spring's independent architecture check also rejected the controlled violation.

The mutation adds ExpressionUtils.evaluate(String), which creates a SpelExpressionParser. Three tests cover arithmetic, a string operation, and a null result. Repair removes that method and its three tests; it restores the original source dependency edge set.

Open the consolidated repository page

Repository selection and architectural basis

Historical selection evidence and scope of the experiment

Source revision: 49adc91c7e41eaafb52fc11e1210989894bda1f3. The fork baseline, 30d8786e, is a later commit containing the CI setup. These revisions serve different purposes.

Selection method. A saved GitHub GraphQL query selected Java repositories with more than 5,000 stars and at least 1,000 forks, excluding forks and archived repositories. The architecture-in-README filter reduced the historical pool from 500 to 77 repositories; the first 20 were retrieved in descending star order.

Spring ranked third in that snapshot (60,293 stars; 38,752 forks). The first candidate was an examples collection; the second used architecture in a CPU/platform context. Spring was the first suitable framework candidate after checking its package constraints and buildable module. This was a popularity-ordered selection, not random sampling or a quality ranking.

Maintainer basis. Spring's ArchitectureRules.java groups packages by their first four segments and prohibits cycles between the slices. ExpressionUtils.java describes Common utilities usable by expression-language providers.

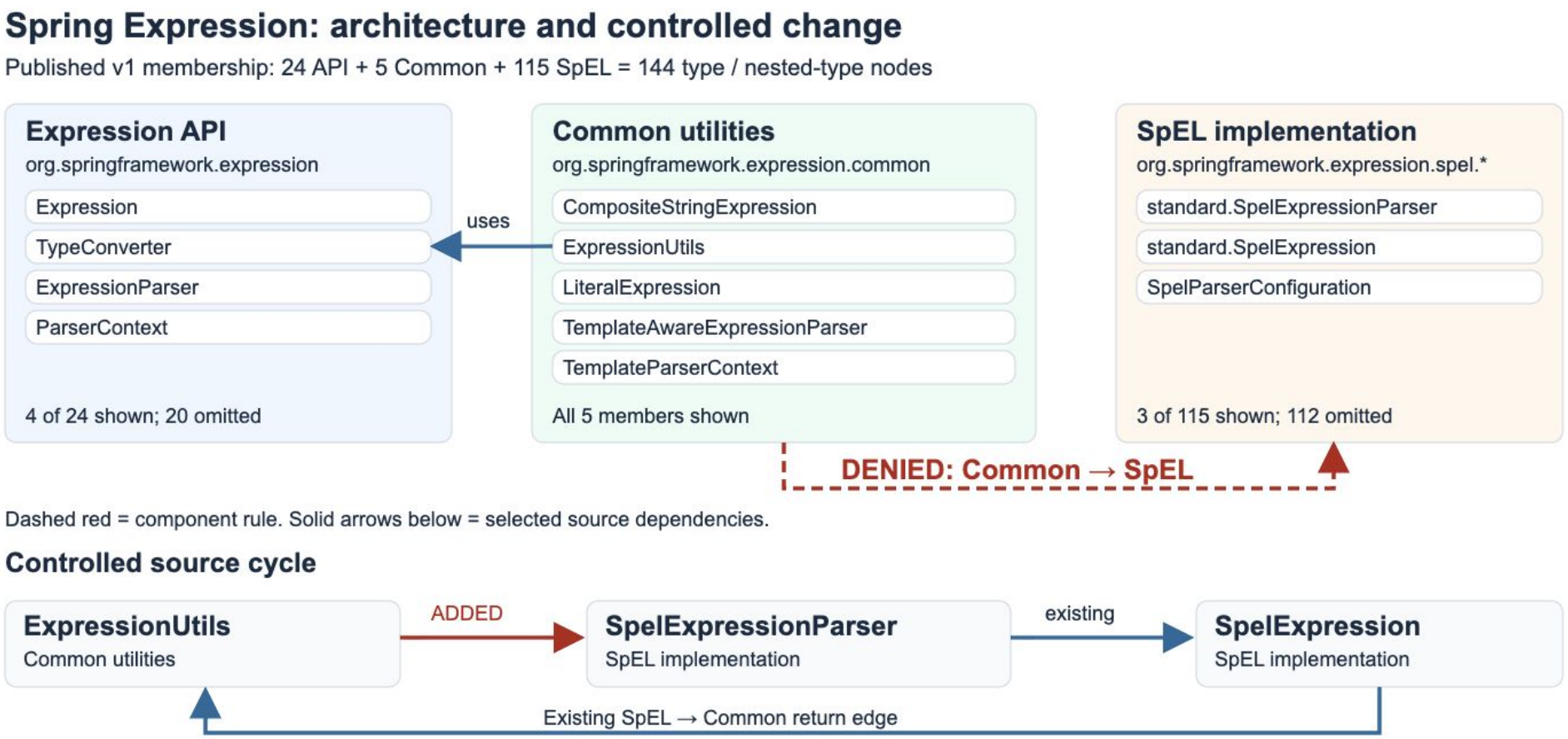
Experimental rule. Existing SpEL-to-Common dependencies make a reverse Common-to-SpEL dependency a controlled way to create a cycle. DCode marks Common utilities to SpEL implementation as **Denied**. This operationalizes the acyclic-slice constraint for this experiment; it is not a literal directional rule quoted from Spring's maintainers.

Saved selection query, variables, metadata and shortlist

Maintainer-authored ArchitectureRules.java

Detailed intended architecture

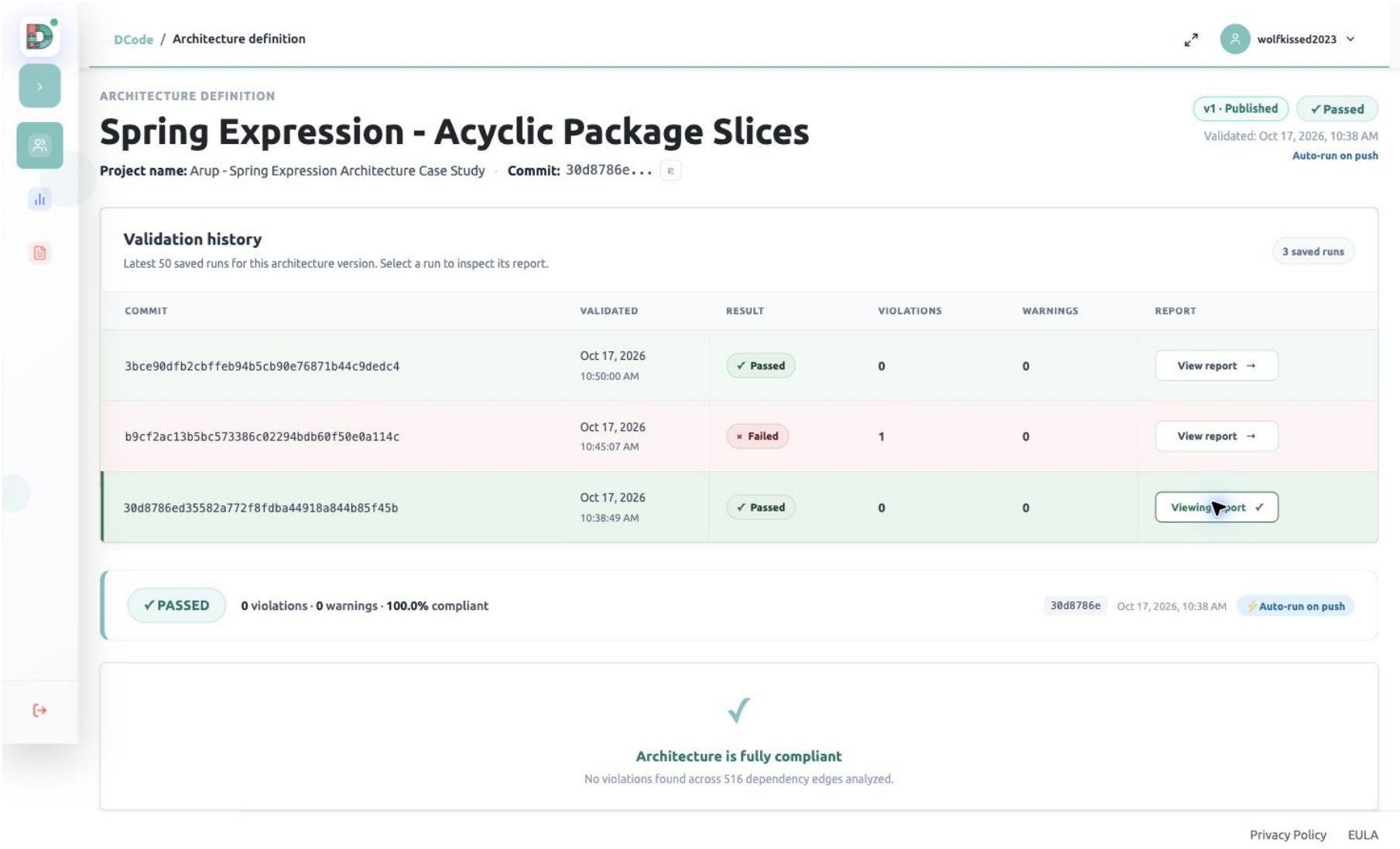
Published v1 membership and the verified source cycle; representative API and SpEL nodes



Reconstruction from original membership and GraphML, not a UI screenshot. Dashed red: component rule. Solid red: added source edge. Blue: existing source dependencies. The full graph contains other edges. All 144 nodes are preserved in the evidence bundle.

Baseline: passed

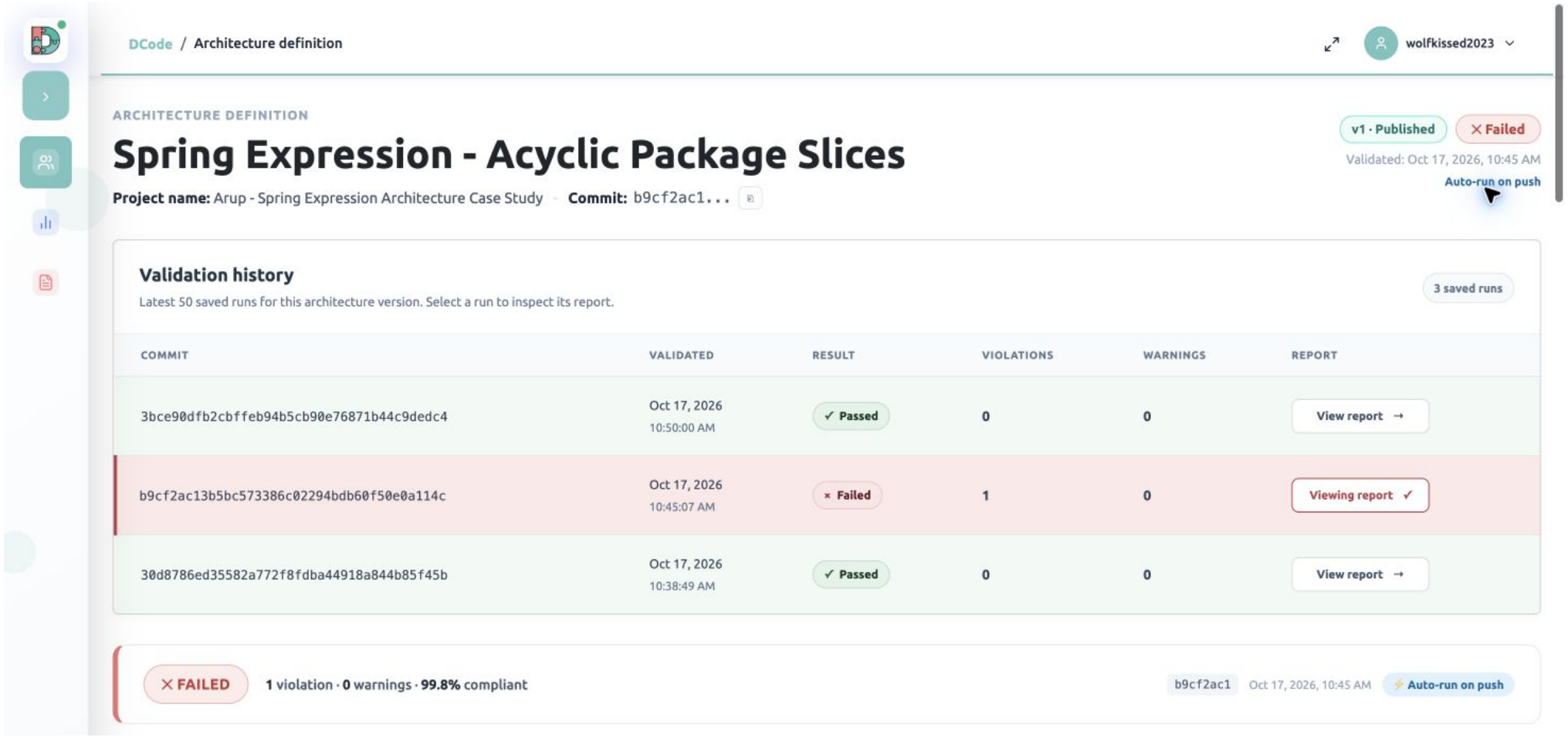
Commit 30d8786e | Published v1 | 0 violations | 516 analyzed dependency edges



Original saved DCode run, opened on 27 October 2026. The validation timestamp remains 17 October. No new analysis was executed for this capture.

Controlled violation: failed

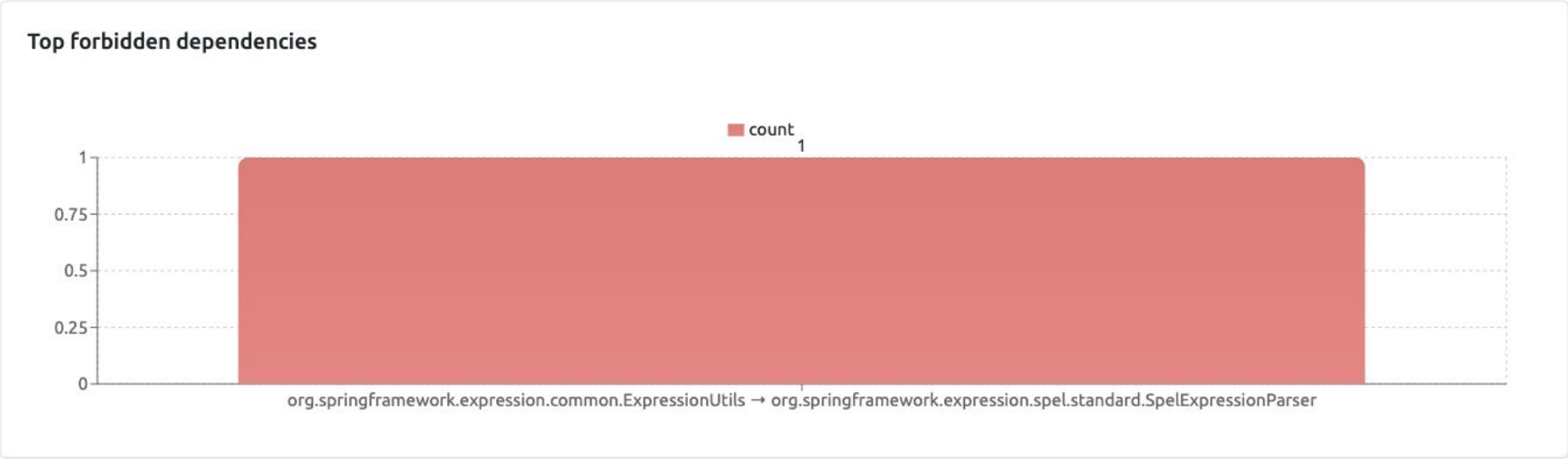
Commit b9cf2ac1 | Published v1 | 1 violation | 517 analyzed dependency edges



Overview excerpt from the saved report. A 99.8% aggregate compliance rate does not make the gate pass: one denied dependency is sufficient for failure.

The violated dependency and rule

Commit b9cf2ac1 | Published v1 | Detail excerpt from the same saved report



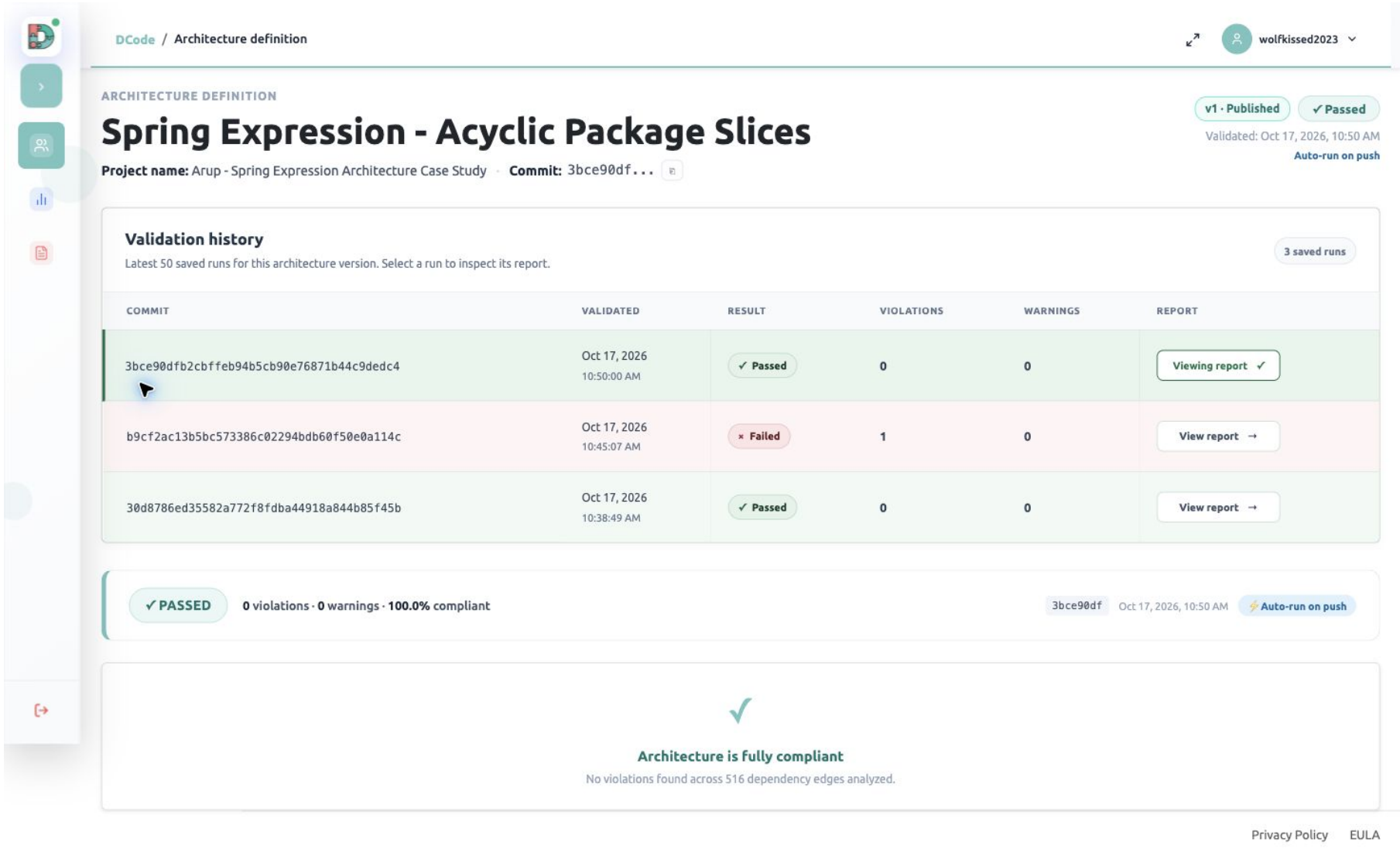
Rule breakdown

#	RULE	ENFORCEMENT	COUNT	% OF TOTAL
1	Common utilities → SpEL implementation layer-edge-1792243993466	Violation	1	100.0%

Lower portion of the same saved report. ExpressionUtils to SpelExpressionParser crosses the denied Common-to-SpEL boundary. The middle statistical charts are omitted from this PDF excerpt.

Repair: passed

Commit 3bce90df | Published v1 | 0 violations | 516 analyzed dependency edges



Removing the controlled mutation restores the original dependency edge set. This is the saved repair report, not a new validation run.

Audit the recorded evidence

Public files; no DCode account or analyzer required

The repository now preserves a small evidence subset for each original CI stage: **DCode result JSON, dependency GraphML, upstream-check logs and original JUnit XML files**. The violation stage also includes Spring's architecture failure report.

The XML files were repacked into ZIP archives without changing their bytes. The provenance file records full commits, original artifact names and member paths, individual XML hashes, and derived test totals. Binary caches and unrelated analyzer outputs are omitted.

Offline verification. From the current documentation checkout, run:

```
python3 case-study/paper-materials/evidence/verify.py
```

The standard-library verifier checks the bundle checksum manifest and XML hashes, recounts tests, verifies all 144 membership nodes, checks active gate results, and compares the three edge sets. It also checks each source edge depicted in the cycle and the ExpressionUtils-to-TypeConverter edge.

Expected result: 1,074 / 1,077 / 1,074 passed tests; 11 skipped each; zero failures and errors; 516 / 517 / 516 edges; PASSED / FAILED / PASSED. There is exactly one added edge, no removed edges in the mutation, and exact baseline edge-set restoration after repair.

This is an audit of preserved records, not a fresh execution. Checksums establish integrity against the supplied manifest; they are not independent proof of the original execution.

Open evidence bundle and verifier

Inspect the controlled source diff

Repeat the recorded stages

Use a separate clone and the pinned historical commits

Use **Temurin JDK 25** and the checked-in Gradle wrapper. The original workflow used ubuntu-latest, which is not an immutable runner. Dependency downloads require network access.

git clone https://github.com/beaprogram/spring-framework.git spring-expression-reproduction
cd spring-expression-reproduction
git checkout --detach <one full commit below>

Baseline: 30d8786ed35582a772f8fdba44918a844b85f45b

Controlled violation: b9cf2ac13b5bc573386c02294bdb60f50e0a114c

Repair: 3bce90dfb2cbffeb94b5cb90e76871b44c9dedc4

At each commit, run:
./gradlew :spring-expression:test --no-daemon --max-workers=2
./gradlew :spring-expression:check --no-daemon --max-workers=2

The functional tests should pass in all stages. The independent upstream architecture check should pass in baseline and repair, and fail on the controlled package-slice cycle in the violation stage. That expected failure is separate from the DCode gate verdict.

For graph extraction, the workflow pins DesigniteJava by SHA-256:
8324e6d1a5fc706295a3f46d3124c69e458315ebfbe62c1e364f48e2ca45e73b
Its download URL is mutable. A changed binary must fail verification; access to the matching binary and license/trial capacity is needed.

Recorded workflow

Hosted validation and limitations

Keep the architecture definition fixed; interpret the result within its scope

Fresh DCode validation. In your own authorized project, configure DCODE_API_KEY as a secret and DCODE_PROJECT_ID and DCODE_ENFORCE_ARCH as variables. A different fork must adapt the workflow's repository guard in its own copy. Never commit credentials.

If bootstrap is needed, use enforcement false only to establish the graph. Recreate the exact three component memberships and denied Common-to-SpEL rule, publish the definition, then set enforcement true for all recorded stages. A separate project may use a different version number; match the contents. The workflow does not automatically import the membership JSON.

Upload each actual commit SHA. Inspect dcode-result.json for **is_active: true**, the expected verdict, violations and edge count. A successful upload with an inactive gate is not a passing architecture result. The workflow preserves evidence on gate failure and enforces independently recorded upstream checks.

Bounded conclusion. One module, one fixed rule set and one deliberate mutation demonstrate that passing functional tests can coexist with a rejected architecture change. Repair removes the added API; it is not a behavior-preserving redesign. This does not measure detection accuracy on natural defects or establish whole-framework compliance.

The graph covers Java production-source dependencies, excluding external libraries, runtime edges, Kotlin and test classes. Other modules are build dependencies only. Source files, analyzer rows, graph nodes and dashboard class counts are different measures. Fresh execution also depends on the available service, analyzer, runner and dependencies; identical present-day results are not guaranteed.

Baseline CI run

Violation CI run

Repair CI run

Canonical case study and public evidence